Assignment 4

Recommended readings: **Lecture slides and MDN links as starting literature**

Note: All exercises must be solved with plain JavaScript and CSS. Using additional libraries or frameworks is not allowed.

Exercise 1 – Working with Closures 0,2 Points

Open the enclosed file `productsA.html` in a browser. The file `shopA.js` already contains some code for a simplistic shopping cart. You must not change the HTML file. Explain the code of `shopA.js` in detail and answer the following additional questions:

a)
   a. Why can the function `addListener` access the current product (cp) when executed?
   b. What happens, if `let cp = {};` in line 21 is replaced by `var cp = {};`?
      Explain scopes in JavaScript and why the behavior changes.

b) Transform the function `addListener()` to an anonymous function, which is directly embedded in `clickButton.addEventListener('click', …);`
   a. Use a standard anonymous function (function (..)).
   b. Use an Arrow Function.

c) Extend the code for making the button “Empty Shopping Cart” work.

d) Extend the code such that all products on the web page are sorted in ascending order of the price attribute. Only change the JS file!

Exercise 2 – JavaScript Objects + this 0,2 Points

Open the enclosed file `productsB.html` in a browser. The file `shopB.js` contains an advanced version of `productsA.js`. However, the code is not working as expected.

a) Explain the code of `shopB.js` in detail and answer the following questions.
   a. Why is the shopping cart not working? Resolve the problem(s).
   b. How often will the code for the function `Product.render()` be in memory when 4 objects are instantiated?

b) Create a new version of `productsC.js`, including all methods from `shopB.js`. This time, define all functions via the **prototype of `Product`.**

a. Add an additional button (via JavaScript) for decrementing the number of items (enabled if number of items is larger than 0). Add the corresponding event handler and a method `removeOneFromCart()` via the prototype.
b. Add an additional method to the prototype that calculates the total item price. Change the shopping cart rendering function to show the total price of each item.
c. Change the render function of only the first item (Sony A7V) such that additionally the text "New Arrival!" is rendered as a postfix.

Exercise 3 – Closures and Currying 0,2 Points

---

a) What is a higher order function? Create your own executable example for a higher-order function. Provide three examples where higher order functions are used in JavaScript. What are first class functions?
b) What is a closure in JavaScript? Create your own example that demonstrates the principle of a closure.
c) What is currying? Provide your own executable example for a curried function.
d) Starting from `shopB.js`, create a new method `getEventHandler()` in the constructor, returning a function that actually calls the `updateProduct()` method. Adapt the code clickButton.addEventListener(…) accordingly. Tip: Return an arrow function! Why is this important?
e) Given the following code:

```
<html>
<script>
function myFunct(a,b,x) {
    let calc = function (i) {
        console.log(i+b);
        return i+b;
    }
    if (a < 20) return calc(a);
    else {
        console.log(a);
        return calc;
    }
}
let x = myFunct(12,8);
let y = myFunct(21,7);
y(14);
y(9,33);
</script>
```

What will be printed on the console and why?

Exercise 4 – JavaScript Prototypes 0,2 Points

a) Create an object for an image that sets width, height, and bitdepth (number of bits used to save color information for every pixel) by using the JSON syntax. Extend the object by a function `computeSize()`, which computes the raw size needed for the image (in bytes) and stores it in the instance variable `rawsize`. Furthermore, the function should also compute the number of pixels and save it in the instance variable `pixels`. Call the function for the object and print the values of both instance variables to the console.

b) Define a constructor function `Image` that allows creating objects like above (with `width`, `height`, and `bitdepth`). Instead of a `computeSize()` function, set `rawsize` and `pixels` already in the constructor function. Additionally, define a function `print()` within the `Image` constructor function. The function should print all properties and values of the object (using a for loop) to the console. Create a few objects to test the print function.

c) Extend the prototype of Image with another function `printMore()`, which prints everything `print()` does, but also if the image is portrait or landscape, depending on the ratio between width and height. Test your implementation with the objects created in Exercise 4b.

d) What is the difference between `print()` and `printMore()`? Can you make it refer to different functions during runtime by means of assignment?

e) Create another constructor function `Video`, which requires width, height, bitdepth, duration (in seconds), and framerate (in frames per second; a 'frame' is one image in the video). `Video` inherits from `Image` (i.e., you should call the constructor function of `Image`). In addition to `Image` it also has a function `totalFrames()`, which returns (based on duration and framerate) the total number of frames in that video. Test your implementation by creating a few objects, calling `print()`, and checking the return value of `totalFrames()`.

f) Can you also call `printMore()` on these objects?

Exercise 5 – JavaScript Classes 0,2 Points

Re-implement all functionality of Exercise 4 using ECMAScript 2015 classes instead of prototypes.

Recommended Project Preparations

Consider everything you learned in this assignment and think about objects, classes, and methods you may need.